p0323R6
std::expected

Published Proposal, 2 April 2018

This version:
https://wg21.link/P0323r6

Issue Tracking:
Inline In Spec

Authors:
Vicente Botet (Nokia)
JF Bastien (Apple)

Audience:
LWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/P0323r6.bs

Abstract

Utility class to represent expected object: wording and open questions.

Table of Contents

This paper revises [P0323r5], which applied feedback obtained from LEWG and EWG.

r6 takes in account the LWG feedback in Jacksonville.

- Locate `unexpected` inside `<expected>` header and section.
- Define friend functions for equality operators and `swap`.
- Follows as much as possible `variant` kind of wording.
- Reword `emplace(initializer_list)` overload.
- Use `trait` is `true`/ `false`.
- Simplify Effects clause when there is a simple return.

We have added some additional open points that have not been identified previously after the LWG's feedback:

- Has `unexpected<void>` a sense?
- Missing `expected<T,E>::emplace` for `unexpected<E>`.
- The in-place construction from a `unexpected<E>` using the `unexpect_t` tag doesn't conveys the in-place nature of this constructor.

These should be fixed in another paper if LEWG considered the fix necessary.

[P0323r4] contains motivation, design rationale, implementability information, sample usage, history, alternative designs and related types. r6 and r5 only contain wording and open questions because their purpose is twofold:

- Present appropriate wording for inclusion in the Library Fundamentals TS v3.
- List open questions which the TS should aim to answer.

§ 1. Wording

Below, substitute the `�` character with a number or name the editor finds appropriate for the sub-section.

§ 1.1. `�.�` Expected objects [*expected*]

§ 1.2. `�.�.1` In general [*expected.general*]

This subclause describes class template `expected` that represents expected objects. An `expected<T, E>` object holds an object of type `T` or an object of type `unexpected<E>` and manages the lifetime of the contained objects.

§ 1.3. `�.�.2` Header `<experimental/expected>` synopsis [*expected.synop*]

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {
        // 𝟐.𝟐.4, class template expected
    template <class T, class E>
        class expected;

    // 𝟐.𝟐.5, class template unexpected
    template <class E>
        class unexpected;
        template <class E>
          unexpected(E) -> unexpected<E>;

    // 𝟐.𝟐.6, class bad_expected_access
    template <class E>
       class bad_expected_access;

    // 𝟐.𝟐.7, Specialization for void
    template <>
       class bad_expected_access<void>;

    // 𝟐.𝟐.8, unexpect tag
    struct unexpect_t {
       explicit unexpect_t() = default;
    };
    inline constexpr unexpect_t unexpect{};

}}}
```

A program that instantiates the definition of `unexpected` for a non-object type, a function type, an array object, an `unexpected<G>` object type or an object type cv-qualified is ill-formed.

A program that instantiates the definition of template `expected<T, E>` for a reference type or for possibly cv-qualified types `in_place_t`, `unexpect_t` or `unexpected<E>` for the `T` parameter or for a reference type or for possibly cv-qualified `void` type for the `E` parameter is ill-formed.

§ 1.4. 𝟐.𝟐.3 Definitions [*expected.defs*]

An instance of `expected<T, E>` is said to be valued if it contains a object of type `T`. An instance of `expected<T, E>` is said to be disapointed if it contains an object of type `unexpected<E>`.

§ 1.5. 𝟐.𝟐.4 Class template expected [*expected.expected*]

```
template <class T, class E>
class expected {
public:
    using value_type = T;
    using error_type = E;
    using unexpected_type = unexpected<E>;

    template <class U>
    using rebind = expected<U, error_type>;

    // 𝟐.𝟐.4.1, constructors
    constexpr expected();
    constexpr expected(const expected&);
    constexpr expected(expected&&) noexcept(see below);
    template <class U, class G>
        EXPLICIT constexpr expected(const expected<U, G>&);
    template <class U, class G>
        EXPLICIT constexpr expected(expected<U, G>&&);
```

```
    template <class U = T>
        EXPLICIT constexpr expected(U&& v);

    template <class G = E>
        constexpr expected(const unexpected<G>&);
    template <class G = E>
        constexpr expected(unexpected<G> &&);

        template <class... Args>
        constexpr explicit expected(in_place_t, Args&&...);
    template <class U, class... Args>
        constexpr explicit expected(in_place_t, initializer_list<U>, Args&&...);
    template <class... Args>
        constexpr explicit expected(unexpect_t, Args&&...);
    template <class U, class... Args>
        constexpr explicit expected(unexpect_t, initializer_list<U>, Args&&...);


    // ?.?.4.2, destructor
    ~expected();

    // ?.?.4.3, assignment
    expected& operator=(const expected&);
    expected& operator=(expected&&) noexcept(see below);
    template <class U = T> expected& operator=(U&&);
    template <class G = E>
        expected& operator=(const unexpected<G>&);
    template <class G = E>
        expected& operator=(unexpected<G>&&);

        // ?.?.4.4, modifiers

    template <class... Args>
        T& emplace(Args&&...);
    template <class U, class... Args>
        T& emplace(initializer_list<U>, Args&&...);

    // ?.?.4.5, swap
    void swap(expected&) noexcept(see below);

    // ?.?.4.6, observers
    constexpr const T* operator ->() const;
    constexpr T* operator ->();
    constexpr const T& operator *() const&;
    constexpr T& operator *() &;
    constexpr const T&& operator *() const &&;
    constexpr T&& operator *() &&;
    constexpr explicit operator bool() const noexcept;
    constexpr bool has_value() const noexcept;
    constexpr const T& value() const&;
    constexpr T& value() &;
    constexpr const T&& value() const &&;
    constexpr T&& value() &&;
    constexpr const E& error() const&;
    constexpr E& error() &;
    constexpr const E&& error() const &&;
    constexpr E&& error() &&;
    template <class U>
        constexpr T value_or(U&&) const&;
    template <class U>
        constexpr T value_or(U&&) &&;

    // ?.?.4.7, Expected equality operators
    template <class T1, class E1, class T2, class E2>
        friend constexpr bool operator==(const expected<T1, E1>& x, const expected<T2, E2>&
y);
    template <class T1, class E1, class T2, class E2>
```

```
        friend constexpr bool operator!=(const expected<T1, E1>& x, const expected<T2, E2>&
 y);

    // 𝔅.𝔅.4.8, Comparison with T
    template <class T1, class E1, class T2>
                friend constexpr bool operator==(const expected<T1, E1>&, const T2&);
    template <class T1, class E1, class T2>
                friend constexpr bool operator==(const T2&, const expected<T1, E1>&);
    template <class T1, class E1, class T2>
                friend constexpr bool operator!=(const expected<T1, E1>&, const T2&);
    template <class T1, class E1, class T2>
                friend constexpr bool operator!=(const T2&, const expected<T1, E1>&);

    // 𝔅.𝔅.4.9, Comparison with unexpected<E>
    template <class T1, class E1, class E2>
                friend constexpr bool operator==(const expected<T1, E1>&, const unexpected<
E2>&);
    template <class T1, class E1, class E2>
                friend constexpr bool operator==(const unexpected<E2>&, const expected<T1,
E1>&);
    template <class T1, class E1, class E2>
                friend constexpr bool operator!=(const expected<T1, E1>&, const unexpected<
E2>&);
    template <class T1, class E1, class E2>
                friend constexpr bool operator!=(const unexpected<E2>&, const expected<T1,
E1>&);

    // 𝔅.𝔅.4.10, Specialized algorithms
    template <class T1, class E1>
                friend void swap(expected<T1, E1>&, expected<T1, E1>&) noexcept(see below);


private:
    bool has_val; // exposition only
    union
    {
        value_type val; // exposition only
        unexpected_type unexpect; // exposition only
    };
};
```

Any instance of `expected<T, E>` at any given time either contains a value of type `T` or a value of type `unexpected<E>` within their own storage. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the object of type `T` or the object of type `unexpected<E>`. These objects shall be allocated in a region of the `expected<T, E>` storage suitably aligned for the types `T` and `unexpected<E>`. Members `has_val`, `val` and `unexpect` are provided for exposition only. `has_val` indicates whether the `expected<T, E>` object is valued.

`T` shall be `void` or shall be an object type that is not array and shall satisfy the requirements of `Destructible` (Table 27).

`E` shall be object type that is not array and shall satisfy the requirements of `Destructible` (Table 27).

§ 1.6. 𝔅.𝔅.4.1 Constructors [*expected.object.ctor*]

```
constexpr expected();
```

Effects: Value-Initializes `val` if `T` is not `void`.

Postconditions: `bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: If value-initialization of `T` is a constexpr constructor or `T` is `void` this constructor shall be constexpr. This constructor shall not participate in overload resolution unless `is_default_constructible_v<T>` is `true` or `T` is `void`.

```
constexpr expected(const expected& rhs);
```

Effects: If `bool(rhs)`, initializes `val` as if direct-non-list-initializing an object of type `T` with the expression `*rhs` if `T` is not `void`.

If `!bool(rhs)`, initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `unexpected(rhs.error())`.

Postconditions: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructors of `T` or `E`.

Remarks: This constructor shall be defined as deleted unless `is_copy_constructible_v<T>` is `true` or `T` is `void` and `is_copy_constructible_v<E>` is `true`. If `is_trivially_copy_constructible_v<T>` is `true` or `T` is `void` and `is_trivially_copy_constructible_v<E>` is `true`, this constructor shall be a constexpr constructor.

```
constexpr expected(expected && rhs) noexcept(see below);
```

Effects: If `bool(rhs)`, initializes `val` as if direct-non-list-initializing an object of type `T` with the expression `std::move(*rhs)` (if `T` is not `void`).

If `!bool(rhs)`, initializes `val` as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `unexpected(std::move(rhs.error()))`.

`bool(rhs)` is unchanged.

Postconditions: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T` or `E`.

Remarks: The expression inside `noexcept` is equivalent to: `is_nothrow_move_constructible_v<T>` is `true` or `T`is `void` and `is_nothrow_move_constructible_v<E>` is `true`. This constructor shall not participate in overload resolution unless `is_move_constructible_v<T>` is `true` and `is_move_constructible_v<E>` is `true`. If `is_trivially_move_constructible_v<T>` is `true` or `T` is `void` and `is_trivially_move_constructible_v<E>` is `true`, this constructor shall be a constexpr constructor.

```
    template <class U, class G>
    EXPLICIT constexpr expected(const expected<U, G>& rhs);
```

Effects: If `bool(rhs)`, initializes `val` as if direct-non-list-initializing an object of type `T` with the expression `*rhs`, if `T` is not `void`.

If `!bool(rhs)` initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `unexpected(rhs.error())`.

Postconditions: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T` or `E`.

Remarks: This constructor shall not participate in overload resolution unless: `T` and `U` are `void` or

- `is_constructible_v<T, const U&>` is `true` ,
- `is_constructible_v<T, expected<U, G>&>` is `false`,
- `is_constructible_v<T, expected<U, G>&&>` is `false`,
- `is_constructible_v<T, const expected<U, G>&>` is `false`,
- `is_constructible_v<T, const expected<U, G>&&>` is `false`,

- `is_convertible_v<expected<U, G>&, T>` is `false`,
- `is_convertible_v<expected<U, G>&&, T>` is `false`,
- `is_convertible_v<const expected<U, G>&, T>` is `false` and
- `is_convertible_v<const expected<U, G>&&, T>` is `false`

and

- `is_constructible_v<E, const G&>` is `true`,
- `is_constructible_v<unexpected<E>, expected<U, G>&>` is `false`,
- `is_constructible_v<unexpected<E>, expected<U, G>&&>` is `false`,
- `is_constructible_v<unexpected<E>, const expected<U, G>&>` is `false`,
- `is_constructible_v<unexpected<E>, const expected<U, G>&&>` is `false`,
- `is_convertible_v<expected<U, G>&, unexpected<E>>` is `false`,
- `is_convertible_v<expected<U, G>&&, unexpected<E>>` is `false`,
- `is_convertible_v<const expected<U, G>&, unexpected<E>>` is `false` and
- `is_convertible_v<const expected<U, G>&&, unexpected<E>>` is `false`.

The constructor is explicit if and only if

- `T` and `U` are not `void` and `is_convertible_v<U const&, T>` is `false` or
- `is_convertible_v<const G&, E>` is `false`.

```
template <class U, class G>
    EXPLICIT constexpr expected(expected<U, G>&& rhs);
```

Effects: If `bool(rhs)` initializes `val` as if direct-non-list-initializing an object of type `T` with the expression `std::move(*rhs)` or nothing if `T` is `void`.

If `!bool(rhs)`, initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `unexpected(std::move(rhs.error()))`.

`bool(rhs)` is unchanged.

Postconditions: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by operations specified in the effect clause.

Remarks: This constructor shall not participate in overload resolution unless: `T` and `U` are `void` or

- `is_constructible_v<T, U&&>` is `true`,
- `is_constructible_v<T, expected<U, G>&>` is `false`,
- `is_constructible_v<T, expected<U, G>&&>` is `false`,
- `is_constructible_v<T, const expected<U, G>&>` is `false`,
- `is_constructible_v<T, const expected<U, G>&&>` is `false`,
- `is_convertible_v<expected<U, G>&, T>` is `false`,
- `is_convertible_v<expected<U, G>&&, T>` is `false`,
- `is_convertible_v<const expected<U, G>&, T>` is `false`, and
- `is_convertible_v<const expected<U, G>&&, T>` is `false`.

and

- `is_constructible_v<E, G&&>` is `true`,
- `is_constructible_v<unexpected<E>, expected<U, G>&>` is `false`,
- `is_constructible_v<unexpected<E>, expected<U, G>&&>` is `false`,
- `is_constructible_v<unexpected<E>, const expected<U, G>&>` is `false`,

- `is_constructible_v<unexpected<E>, const expected<U, G>&&>` is `false`,
- `is_convertible_v<expected<U, G>&, unexpected<E>>` is `false`,
- `is_convertible_v<expected<U, G>&&, unexpected<E>>` is `false`,
- `is_convertible_v<const expected<U, G>&, unexpected<E>>` is `false` and
- `is_convertible_v<const expected<U, G>&&, unexpected<E>>` is `false`.

The constructor is explicit if and only if

- `T` and `U` are not `void` and `is_convertible_v<U&&, T>` is `false` or
- `is_convertible_v<G&&, E>` is `false`.

```
template <class U = T>
    EXPLICIT constexpr expected(U&& v);
```

Effects: Initializes `val` as if direct-non-list-initializing an object of type `T` with the expression `std::forward<U>(v)`.

Postconditions: `bool(*this)` .

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: If `T`'s selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless

- `T` is not `void`,
- `is_constructible_v<T, U&&>` is `true`,
- `is_same_v<remove_cvref_t<U>, in_place_t>` is `false`,
- `is_same_v<expected<T, E>, remove_cvref_t<U>>` is `false` and
- `is_same_v<unexpected<E>, remove_cvref_t<U>>` is `false`.

The constructor is explicit if and only if `is_convertible_v<U&&, T>` is `false`.

```
template <class G = E>
    EXPLICIT constexpr expected(const unexpected<G>& e);
```

Effects: Initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `e`.

Postconditions: `!bool(*this)`.

Throws: Any exception thrown by the selected constructor of `E`.

Remark: If `unexpected<E>`'s selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<E, const G&>` is `true`. The constructor is explicit if and only if `is_convertible_v<const G&, E>` is `false`.

```
template <class G = E>
    EXPLICIT constexpr expected(unexpected<G>&& e);
```

Effects: Initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with the expression `std::move(e)`.

Postconditions: `!bool(*this)`.

Throws: Any exception thrown by the selected constructor of `E`.

Remark: If `unexpected<E>`'s selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. The expression inside `noexcept` is equivalent to: `is_nothrow_constructible_v<E,`

`G&&>` is `true`. This constructor shall not participate in overload resolution unless `is_constructible_v<E, G&&>` is `true`. The constructor is explicit if and only if `is_convertible_v<G&&, E>` is `false`.

```
template <class... Args>
    constexpr explicit expected(in_place_t, Args&&... args);
```

Effects: Initializes `val` as if direct-non-list-initializing an object of type `T` with the arguments `std::forward<Args>(args)...` if `T` is not `void`.

Postconditions: `bool(*this)` .

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: If `T`'s constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `T` is `void` and `sizeof...(Args) == 0` or `T` is not `void` and `is_constructible_v<T, Args...>` is `true`.

```
template <class U, class... Args>
    constexpr explicit expected(in_place_t, initializer_list<U> il, Args&&... args);
```

Effects: Initializes `val` as if direct-non-list-initializing an object of type `T` with the arguments `il, std::forward<Args>(args)...`.

Postconditions: `bool(*this)` .

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: If `T`'s constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `T` is not `void` and `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`.

```
template <class... Args>
    constexpr explicit expected(unexpect_t, Args&&... args);
```

Effects: Initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>`with the arguments `std::forward<Args>(args)...`.

Postconditions: `!bool(*this)`.

Throws: Any exception thrown by the selected constructor of `E`.

Remarks: If `unexpected<E>`'s constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<E, Args&&...>` is `true`.

```
template <class U, class... Args>
    constexpr explicit expected(unexpect_t, initializer_list<U> il, Args&&... args);
```

Effects: Initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with the arguments `il, std::forward<Args>(args)...`.

Postconditions: `!bool(*this)`.

Throws: Any exception thrown by the selected constructor of `E`.

Remarks: If `unexpected<E>`'s constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<E, initializer_list<U>&, Args&&...>` is `true`.

§ 1.7. 🔒.🔒.4.2 Destructor [*expected.object.dtor*]

```
~expected();
```

Effects: If `T` is not `void` and `is_trivially_destructible_v<T>` is `false` and `bool(*this)`, calls `val.~T()`. If `is_trivially_destructible_v<E>` is `false` and `!bool(*this)`, calls `unexpect.~unexpected<E>()`.

Remarks: If `T` is `void` or `is_trivially_destructible_v<T>` is `true` and `is_trivially_destructible_v<E>` is `true` then this destructor shall be a trivial destructor.

1.8. 𝔵.𝔵.4.3 Assignment [*expected.object.assign*]

```
expected& operator=(const expected& rhs) noexcept(see below);
```

Effects:

If `bool(*this)` and `bool(rhs)`,

- assigns `*rhs` to `val` if `T` is not `void`;

otherwise if `!bool(*this)` and `!bool(rhs)`,

- assigns `unexpected(rhs.error())` to `unexpect`;

otherwise if `bool(*this)` and `!bool(rhs)`,

- if `T` is `void`
  - initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(rhs.error())`. Either
  - the constructor didn't throw, set `has_val` to `false`, or
  - the constructor did throw, and nothing was changed.
- otherwise if `is_nothrow_copy_constructible_v<E>` is `true`
  - destroys `val` by calling `val.~T()`,
  - initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(rhs.error())` and set `has_val` to `false`.
- otherwise if `is_nothrow_move_constructible_v<E>` is `true`
  - constructs a `unexpected<E> tmp` from `unexpected(rhs.error())` (this can throw),
  - destroys `val` by calling `val.~T()`,
  - initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with `std::move(tmp)` and set `has_val` to `false`.

otherwise

- constructs a `T tmp` from `*this` (this can throw),
- destroys `val` by calling `val.~T()`,
- initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(rhs.error())`. Either,
  - the last constructor didn't throw, set `has_val` to `false`, or
  - the last constructor did throw, so move-construct the `T` from `tmp` back into the expected storage (which can't throw as `is_nothrow_move_constructible_v<T>` is `true`), and rethrow the exception.

otherwise

- if `T` is `void` destroys `unexpect` by calling `unexpect.~unexpected<E>()` and set `has_val` to `true`,
- otherwise if `is_nothrow_copy_constructible_v<T>` is `true`

    - destroys `unexpect` by calling `unexpect.~unexpected<E>()`
    - initializes `val` as if direct-non-list-initializing an object of type `T` with `*rhs`;
- otherwise if `is_nothrow_move_constructible_v<T>` is `true`
    - constructs a `T tmp` from `*rhs` (this can throw),
    - destroys `unexpect` by calling `unexpect.~unexpected<E>()`
    - initializes `val` as if direct-non-list-initializing an object of type `T` with `std::move(tmp)`;
- otherwise
    - constructs a `unexpected<E> tmp` from `unexpected(this->error())` (which can throw),
    - destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
    - initializes `val` as if direct-non-list-initializing an object of type `T` with `*rhs`. Either,
    - the constructor didn't throw, set `has_val` to `true`, or
    - the constructor did throw, so move-construct the `unexpected<E>` from `tmp` back into the expected storage (which can't throw as `is_nothrow_move_constructible_v<E>` is `true`), and rethrow the exception.

Returns: `*this`.

Postconditions: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the operations specified in the effect clause.

Remarks: If any exception is thrown, `bool(*this)` and `bool(rhs)` remain unchanged.

If an exception is thrown during the call to `T`'s or `unexpected<E>`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s or `unexpected<E>`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s or `unexpected<E>`'s copy assignment.

This operator shall be defined as deleted unless

- `T` is `void` and `is_copy_assignable_v<E>` is `true` and `is_copy_constructible_v<E>` is `true` or
- `T` is not `void` and `is_copy_assignable_v<T>` is `true` and `is_copy_constructible_v<T>` is `true` and `is_copy_assignable_v<E>` is `true` and `is_copy_constructible_v<E>` is `true` and (`is_nothrow_move_constructible_v<E>` is `true` or `is_nothrow_move_constructible_v<T>` is `true`).

```
expected& operator=(expected&& rhs) noexcept(see below);
```

Effects:

If `bool(*this)` and `bool(rhs)`,

- move assign `*rhs` to `val` if `T` is not `void`;

otherwise if `!bool(*this)` and `!bool(rhs)`,

- move assign `unexpected(rhs.error())` to `unexpect`;

otherwise if `bool(*this)` and `!bool(rhs)`,

- if `T` is `void`
    - initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(move(rhs).error())`. Either
    - the constructor didn't throw, set `has_val` to `false`, or
    - the constructor did throw, and nothing was changed.
- otherwise if `is_nothrow_move_constructible_v<E>` is `true`
    - destroys `val` by calling `val.~T()`,

    - initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(std::move(rhs.error()))`;
- otherwise
    - move constructs a `T tmp` from `*this` (which can't throw as `T` is nothrow-move-constructible),
    - destroys `val` by calling `val.~T()`,
    - initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected_type<E>` with `unexpected(std::move(rhs.error()))`. Either,
    - The constructor didn't throw, so mark the expected as holding a `unexpected_type<E>`, or
    - The constructor did throw, so move-construct the `T` from `tmp` back into the expected storage (which can't throw as `T` is nothrow-move-constructible), and rethrow the exception.

otherwise `!bool(*this)` and `bool(rhs)`,

- if `T` is `void` destroys `unexpect` by calling `unexpect.~unexpected<E>()`
- otherwise if `is_nothrow_move_constructible_v<T>` is `true`
    - destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
    - initializes `val` as if direct-non-list-initializing an object of type `T` with `*std::move(rhs)`;
- otherwise
    - move constructs a `unpepected_type<E> tmp` from `unexpected(this->error())` (which can't throw as `E` is nothrow-move-constructible),
    - destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
    - initializes `val` as if direct-non-list-initializing an object of type `T` with `*std::move(rhs)`. Either,
    - The constructor didn't throw, set `has_val` to `true`, or
    - The constructor did throw, so move-construct the `unexpected<E>` from `tmp` back into the expected storage (which can't throw as `E` is nothrow-move-constructible), and rethrow the exception.

Returns: `*this`.

Postconditions: `bool(rhs) == bool(*this)`.

Remarks: The expression inside noexcept is equivalent to: `is_nothrow_move_assignable_v<T>` is `true` and `is_nothrow_move_constructible_v<T>` is `true`.

If any exception is thrown, `bool(*this)` and `bool(rhs)` remain unchanged. If an exception is thrown during the call to `T`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment. If an exception is thrown during the call to `E`'s copy assignment, the state of its contained `unexpect` is as defined by the exception safety guarantee of `E`'s copy assignment.

This operator shall be defined as deleted unless

- `T` is `void` and `is_nothrow_move_constructible_v<E>` is `true` and `is_nothrow_move_assignable_v<E>` is `true`.

or

- `T` is not `void` and `is_move_constructible_v<T>` is `true` and `is_move_assignable_v<T>` is `true` and `is_nothrow_move_constructible_v<E>` is `true` and `is_nothrow_move_assignable_v<E>` is `true`.

```
template <class U = T>
    expected<T, E>& operator=(U&& v);
```

Effects:

If `bool(*this)`, assigns `std::forward<U>(v)` to `val`;

otherwise if `is_nothrow_constructible_v<T, U&&>` is `true`

- destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
- initializes `val` as if direct-non-list-initializing an object of type `T` with `std::forward<U>(v)` and
- set `has_val` to `true`;

otherwise

- move constructs a `unexpected<E> tmp` from `unexpected(this->error())` (which can't throw as `E` is nothrow-move-constructible),
- destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
- initializes `val` as if direct-non-list-initializing an object of type `T` with `std::forward<U>(v)`. Either,
  - the constructor didn't throw, set `has_val` to `true`, that is set `has_val` to `true`, or
  - the constructor did throw, so move construct the `unexpected<E>` from `tmp` back into the expected storage (which can't throw as `E` is nothrow-move-constructible), and re-throw the exception.

Returns: `*this`.

Postconditions: `bool(*this)` .

Remarks: If any exception is thrown, `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment.

This function shall not participate in overload resolution unless:

- `is_void_v<T>` is `false` and
- `is_same_v<expected<T,E>, remove_cvref_t<U>>` is `false` and
- `conjunction_v<is_scalar<T>, is_same<T, decay_t<U>>>` is `false`,
- `is_constructible_v<T, U>` is `true`,
- `is_assignable_v<T&, U>` is `true` and
- `is_nothrow_move_constructible_v<E>` is `true`.

```
template <class G = E>
    expected<T, E>& operator=(const unexpected<G>& e);
```

Effects:

If `!bool(*this)`, assigns `unexpected(e.error())` to `unexpect`;

otherwise

- destroys `val` by calling `val.~T()` if `T` is not `void`,
- initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(e.error())` and set `has_val` to `false`.

Returns: `*this`.

Postconditions: `!bool(*this)`.

Remarks: If any exception is thrown, `bool(*this)` remains unchanged.

This signature shall not participate in overload resolution unless `is_nothrow_copy_constructible_v<E>` is `true` and `is_move_assignable_v<E>` is `true`.

```
expected<T, E>& operator=(unexpected<G> && e);
```

Effects:

If `!bool(*this)`, move assign `unexpected(e.error())` to `unexpect`;

otherwise

- destroys `val` by calling `val.~T()` if `T` is not `void`,
- initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(std::move(e.error()))` and set `has_val` to `false`.

Returns: `*this`.

Postconditions: `!bool(*this)`.

Remarks: If any exception is thrown, `bool(*this)` remains unchanged.

This signature shall not participate in overload resolution unless `is_nothrow_move_constructible_v<E>` is `true` and `is_move_assignable_v<E>` is `true`.

```
void expected<void,E>::emplace();
```

Effects:

If `!bool(*this)`

- destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
- set `has_val` to `true`

Postconditions: `bool(*this)` .

Throws: Nothing

```
template <class... Args>
    T& emplace(Args&&... args);
```

Effects:

If `bool(*this)`, assigns `val` as if constructing an object of type `T` with the arguments `std::forward<Args>(args)...`

otherwise if `is_nothrow_constructible_v<T, Args&&...>` is `true`

- destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
- initializes `val` as if direct-non-list-initializing an object of type `T` with `std::forward<Args>(args)...` and
- set `has_val` to `true`;

otherwise if `is_nothrow_move_constructible_v<T>` is `true`

- constructs a `T tmp` from `std::forward<Args>(args)...` (which can throw),
- destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
- initializes `val` as if direct-non-list-initializing an object of type `T` with `std::move(tmp)` (which cannot throw) and
- set `has_val` to `true`;

otherwise

- move constructs a `unexpected<E> tmp` from `unexpected(this->error())` ,
- destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
- initializes `val` as if direct-non-list-initializing an object of type `T` with `std::forward<Args>(args)...`. Either,
    - the constructor didn't throw, set `has_val` to `true`, or

- the constructor did throw, so move-construct the `unexpected<E>` from `tmp` back into the expected storage (which can't throw as `E` is nothrow-move-constructible), and re-throw the exception.

Postconditions: `bool(*this)` .

Returns: A reference to the new contained value `val`.

Throws: Any exception thrown by the operations specified in the effect clause.

Remarks: If an exception is thrown during the call to `T`'s assignment, nothing changes.

This signature shall not participate in overload resolution unless: `T` is not `void` and `is_nothrow_constructible_v<T, Args&&...>` is `true`.

```
template <class U, class... Args>
    T& emplace(initializer_list<U> il, Args&&... args);
```

Effects:

If `bool(*this)`, assigns `val` as if constructing an object of type `T` with the arguments `il, std::forward<Args>(args)...`

otherwise if `is_nothrow_constructible_v<T, initializer_list<U>&, Args&&...>` is `true`

- destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
- initializes `val` as if direct-non-list-initializing an object of type `T` with `il, std::forward<Args>(args)...` and
- set `has_val` to `true`;

otherwise if `is_nothrow_move_constructible_v<T>` is `true`

- constructs a `T tmp` from `il, std::forward<Args>(args)...` (which can throw),
- destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
- initializes `val` as if direct-non-list-initializing an object of type `T` with `std::move(tmp)` (which cannot throw) and
- set `has_val` to `true`;

otherwise

- move constructs a `unexpected<E> tmp` from `unexpected(this->error())` ,
- destroys `unexpect` by calling `unexpect.~unexpected<E>()`,
- initializes `val` as if direct-non-list-initializing an object of type `T` with `il, std::forward<Args>(args)...`. Either,
  - the constructor didn't throw, set `has_val` to `true`, or
  - the constructor did throw, so move-construct the `unexpected<E>` from `tmp` back into the expected storage (which can't throw as `E` is nothrow-move-constructible), and re-throw the exception.

Postconditions: `bool(*this)` .

Returns: A reference to the new contained value `val`.

Throws: Any exception thrown by the operations specified in the effect clause.

Remarks: If an exception is thrown during the call to `T`'s assignment nothing changes.

The function shall not participate in overload resolution unless: `T` is not `void` and `is_nothrow_constructible_v<T, initializer_list<U>&, Args&&...>` is `true`.

§ 1.9. 𝔼.𝔼.4.4 Swap [*expected.object.swap*]

```
void swap(expected<T, E>& rhs) noexcept(see below);
```

Effects: if `bool(*this)` and `bool(rhs)`,

- if `T` is not `void` calls `using std::swap; swap(val, rhs.val)`,

otherwise if `!bool(*this)` and `!bool(rhs)`,

- calls `using std::swap; swap(unexpect, rhs.unexpect)`,

otherwise if `!bool(*this)` and `bool(rhs)`,

- calls `rhs.swap(*this)`,

otherwise

- if `T` is `void`
  - initializes `unexpect` as if direct-non-list-initializing an object of type `unexpected<E>` with `unexpected(std::move(rhs))`. Either
  - the constructor didn't throw, set `has_val` to `false`, destroys `rhs.unexpect` by calling `rhs.unexpect.~unexpected<E>()` set `rhs.has_val` to `true`.
  - the constructor did throw, rethrow the exception.
- otherwise if `is_nothrow_move_constructible_v<E>` is `true`,
  - the `unexpect` of `rhs` is moved to a temporary variable `tmp` of type `unexpected_type`,
  - followed by destruction of `unexpect` as if by `rhs.unexpect.~unexpected<E>()`,
  - `rhs.val` is direct-initialized from `std::move(*this)`. Either
    - the constructor didn't throw
      - destroy `val` as if by `this->val.~T()`,
      - the `unexpect` of `this` is direct-initialized from `std::move(tmp)`, after this, `this` does not contain a value; and `bool(rhs)`.
    - the constructor did throw, so move-construct the `unexpected<E>` from `tmp` back into the expected storage (which can't throw as `E` is nothrow-move-constructible), and re-throw the exception.
- otherwise if `is_nothrow_move_constructible_v<T>` is `true`,
  - `val` is moved to a temporary variable `tmp` of type `T`,
  - followed by destruction of `val` as if by `val.~T()`,
  - the `unexpect` is direct-initialized from `unexpected(std::move(other.error()))`. Either
    - the constructor didn't throw
      - destroy `rhs.unexpect` as if by `rhs.unexpect.~unexpected<E>()`,
      - `rhs.val` is direct-initialized from `std::move(tmp)`, after this, `this` does not contain a value; and `bool(rhs)`.
    - the constructor did throw, so move-construct the `T` from `tmp` back into the expected storage (which can't throw as `T` is nothrow-move-constructible), and re-throw the exception.
- otherwise, the function is not defined.

Throws: Any exceptions that the expressions in the Effects clause throw.

Remarks: The expression inside noexcept is equivalent to: `is_nothrow_move_constructible_v<T>` is `true` and `is_nothrow_swappable_v<T>` is `true` and `is_nothrow_move_constructible_v<E>` is `true` and `is_nothrow_swappable_v<E>` is `true`. The function shall not participate in overload resolution unless:

- Lvalues of type `T` shall be `Swappable`,
- Lvalues of type `E` shall be `Swappable`,
- `is_void_v<T>` is `true` or
- `is_void_v<T>` is `false` and
  - `is_move_constructible_v<T>` is `true`,
  - `is_move_constructible_v<E>` is `true` and
  - `is_move_constructible_v<E>` is `true` or `is_move_constructible_v<T>` is `true`.

§ 1.10. 𝔵.𝔵.4.5 Observers [*expected.object.observe*]

```
constexpr const T* operator->() const;
T* operator->();
```

Requires: `bool(*this)` .

Returns: `addressof(val)`.

Remarks: Unless `T` is a user-defined type with overloaded unary `operator&`, the first operator shall be a constexpr function. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr const T& operator *() const&;
T& operator *() &;
```

Requires: `bool(*this)` .

Returns: `val`.

Remarks: The first operator shall be a constexpr function. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr T&& operator *() &&;
constexpr const T&& operator *() const&&;
```

Requires: `bool(*this)` .

Returns: `std::move(val)`.

Remarks: This operator shall be a constexpr function. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr explicit operator bool() noexcept;
```

Returns: `has_val`.

Remarks: This operator shall be a constexpr function.

```
constexpr bool has_value() const noexcept;
```

Returns: `has_val`.

Remarks: This function shall be a constexpr function.

```
constexpr void expected<void, E>::value() const;
```

Throws: `bad_expected_access(error())` if `!bool(*this)`.

```
constexpr const T& expected::value() const&;
constexpr T& expected::value() &;
```

Returns: `val`, if `bool(*this)` .

Throws: `bad_expected_access(error())` if `!bool(*this)`.

Remarks: These functions shall be constexpr functions. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr T&& expected::value() &&;
constexpr const T&& expected::value() const&&;
```

Returns: `std::move(val)`, if `bool(*this)` .

Throws: `bad_expected_access(error())` if `!bool(*this)`.

Remarks: These functions shall be constexpr functions. The operator shall not participate in overload resolution unless: `T` is not `void`.

```
constexpr const E& error() const&;
constexpr E& error() &;
```

Requires: `!bool(*this)`.

Returns: `unexpect.value()`.

Remarks: The first function shall be a constexpr function.

```
constexpr E&& error() &&;
constexpr const E&& error() const &&;
```

Requires: `!bool(*this)`.

Returns: `std::move(unexpect.value())`.

Remarks: The first function shall be a constexpr function.

```
template <class U>
    constexpr T value_or(U&& v) const&;
```

Returns: `bool(*this) ? **this : static_cast<T>(std::forward<U>(v));`.

Remarks: If `is_copy_constructible_v<T>` istrue`&& is_convertible_v<U&&, T>` is `false` the program is ill-formed.

```
template <class U>
    constexpr T value_or(U&& v) &&;
```

Returns: `bool(*this) ? std::move(**this) : static_cast<T>(std::forward<U>(v));`.

Remarks: If `is_move_constructible_v<T>` is `true` and `is_convertible_v<U&&, T>` is `false` the program is ill-formed.

§ 1.11. ﷺ.4.6 Expected Equality operators [*expected.equality_op*]

```
template <class T1, class E1, class T2, class E2>
    friend constexpr bool operator==(const expected<T1, E1>& x, const expected<T2, E2>& y);
```

Requires: The expressions `*x == *y` and `unexpected(x.error()) == unexpected(y.error())` shall be well-formed and its result shall be convertible to `bool`.

Returns: If `bool(x) != bool(y)`, `false`; otherwise if `bool(x) == false`, `x.error() == y.error()`; otherwise `true` if `T1` and `T2` are `void` or `*x == *y` otherwise.

Remarks: Specializations of this function template, for which `T1` and `T2` are `void` or `*x == *y` and `x.error() == y.error()` are core constant expression, shall be `constexpr` functions.

```
template <class T1, class E1, class T2, class E2>
    constexpr bool operator!=(const expected<T1, E1>& x, const expected<T2, E2>& y);
```

Requires: The expressions `*x != *y` and `x.error() != y.error()` shall be well-formed and its result shall be convertible to `bool`.

Returns: If `bool(x) != bool(y)`, `true`; otherwise if `bool(x) == false`, `x.error() != y.error()`; otherwise `true` if `T1` and `T2` are `void` or `*x != *y`.

Remarks: Specializations of this function template, for which `T1` and `T2` are `void` or `*x != *y` and `x.error() != y.error()` are core constant expression, shall be constexpr functions.

§ 1.12. 𝔠.𝔠.4.7 Comparison with `T` [*expected.comparison_T*]

```
template <class T1, class E1, class T2> constexpr bool operator==(const expected<T1, E1>& x
, const T2& v);
template <class T1, class E1, class T2> constexpr bool operator==(const T2& v, const expect
ed<T1, E1>& x);
```

Requires: `T1` and `T2` are not `void` and the expression `*x == v` shall be well-formed and its result shall be convertible to `bool`. [*Note:* `T1` need not be *EqualityComparable*. - *end note*]

Returns: `bool(x) ? *x == v : false;`.

```
template <class T1, class E1, class T2> constexpr bool operator!=(const expected<T1, E1>& x
, const T2& v);
template <class T1, class E1, class T2> constexpr bool operator!=(const T2& v, const expect
ed<T1, E1>& x);
```

Requires: `T1` and `T2` are not `void` and the expression `*x == v` shall be well-formed and its result shall be convertible to `bool`. [*Note:* `T1` need not be *EqualityComparable*. - *end note*]

Returns: `bool(x) ? *x != v : false;`.

§ 1.13. 𝔠.𝔠.4.8 Comparison with `unexpected<E>` [*expected.comparison_unexpected_E*]

```
template <class T1, class E1, class E2> constexpr bool operator==(const expected<T1, E1>& x
, const unexpected<E2>& e);
template <class T1, class E1, class E2> constexpr bool operator==(const unexpected<E2>& e,
const expected<T1, E1>& x);
```

Requires: The expression `unexpected(x.error()) == e` shall be well-formed and its result shall be convertible to `bool`.

Returns: `bool(x) ? false : unexpected(x.error()) == e;`.

```
template <class T1, class E1, class E2> constexpr bool operator!=(const expected<T1, E1>& x
, const unexpected<E2>& e);
template <class T1, class E1, class E2> constexpr bool operator!=(const unexpected<E2>& e,
const expected<T1, E1>& x);
```

Requires: The expression `unexpected(x.error()) != e` shall be well-formed and its result shall be convertible to `bool`. *Returns*: `bool(x) ? true : unexpected(x.error()) != e;`.

§ 1.14. 𝟐𝟗.𝟒.4.9 Specialized algorithms [*expected.specalg*]

```
template <class T1, class E1>
void swap(expected<T1, E1>& x, expected<T1, E1>& y) noexcept(noexcept(x.swap(y)));
```

Effects: Calls `x.swap(y)`.

Remarks: This function shall not participate in overload resolution unless `T1` is `void` or `is_move_constructible_v<T1>` is `true`, `is_swappable_v<T1>` is `true` and `is_move_constructible_v<E1>` is `true` and `is_swappable_v<E1>` is `true`.

§ 1.15. 𝟐𝟗.𝟒.5 Unexpected objects [*expected.unexpected*]

§ 1.16. 𝟐𝟗.𝟒.5.1 General [*expected.unexpected.general*]

This subclause describes class template `unexpected` that represents unexpected objects.

§ 1.17. 𝟐𝟗.𝟒.5.2 Class template `unexpected` [*expected.unexpected.object*]

```
template <class E>
class unexpected {
public:
    constexpr unexpected(const unexpected&) = default;
    constexpr unexpected(unexpected&&) = default;
        template <class... Args>
                constexpr explicit unexpected(in_place_t, Args&&...);
    template <class U, class... Args>
                constexpr explicit unexpected(in_place_t, initializer_list<U>, Args&&...);
    template <class Err = E>
                constexpr explicit unexpected(Err&&);
    template <class Err>
                constexpr EXPLICIT unexpected(const unexpected<Err>&);
    template <class Err>
                constexpr EXPLICIT unexpected(unexpected<Err>&&);

    constexpr unexpected& operator=(const unexpected&) = default;
    constexpr unexpected& operator=(unexpected&&) = default;
        template <class Err = E>
    constexpr unexpected& operator=(const unexpected<Err>&);
        template <class Err = E>
    constexpr unexpected& operator=(unexpected<Err>&&);

    constexpr const E& value() const& noexcept;
    constexpr E& value() & noexcept;
    constexpr const E&& value() const&& noexcept;
    constexpr E&& value() && noexcept;

        void swap(unexpected& other) noexcept(see bellow);

    template <class E1, class E2>
        friend constexpr bool
        operator==(const unexpected<E1>&, const unexpected<E2>&);
    template <class E1, class E2>
        constexpr bool
        operator!=(const unexpected<E1>&, const unexpected<E2>&);

    template <class E1>
    friend void swap(unexpected<E1>& x, unexpected<E1>& y) noexcept(noexcept(x.swap(y)));


private:
    E val; // exposition only
};

template <class E> unexpected(E) -> unexpected<E>;
```

..5.2.1 Constructors [*expected.unexpected.ctor*] {#expected.unexpected.ctor}

```
template <class Err>
        constexpr explicit unexpected(Err&& e);
```

Effects: Initializes `val` as if direct-non-list-initializing an object of type `E` with the expression `std::forward<Err>(e)`.

Throws: Any exception thrown by the selected constructor of `E`.

Remarks: If E's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<E, Err&&>` is `true`, `is_same_v<remove_cvref_t<U>, in_place_t>` is `false` and `is_same_v<remove_cvref_t<U>, unexpected>` is `false`.

```
template <class... Args>
        constexpr explicit unexpected(in_place_t, Args&&...);
```

Effects: Initializes `val` as if direct-non-list-initializing an object of type `E` with the arguments `std::forward<Args>(args)...`.

Throws: Any exception thrown by the selected constructor of `E`.

Remarks: If E's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<E, Args&&...>` is `true`.

```
template <class U, class... Args>
        constexpr explicit unexpected(in_place_t, initializer_list<U>, Args&&...);
```

Effects: Initializes `val` as if direct-non-list-initializing an object of type `E` with the arguments `il, std::forward<Args>(args)...`.

Throws: Any exception thrown by the selected constructor of `E`.

Remarks: If E's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<E, initializer_list<U>&, Args&&...>` is `true`.

```
template <class Err>
constexpr EXPLICIT unexpected(const unexpected<Err>& e);
```

Effects: Initializes `val` as if direct-non-list-initializing an object of type `E` with the expression `e.val`.

Throws: Any exception thrown by the selected constructor of `E`.

Remarks: This constructor participates in overload resolution unless

- `std::is_constructible_v<E, Err>` is `true`,
- `is_constructible_v<E, unexpected<Err>&>` is `false`,
- `is_constructible_v<E, unexpected<Err>&&>` is `false`,
- `is_constructible_v<E, const unexpected<Err>&>` is `false`,
- `is_constructible_v<E, const unexpected<Err>&&>` is `false`,
- `is_convertible_v<unexpected<Err>&, E>` is `false`,
- `is_convertible_v<unexpected<Err>&&, E>` is `false`,
- `is_convertible_v<const unexpected<Err>&, E>` is `false`, and
- `is_convertible_v<const unexpected<Err>&&, E>` is `false`.

This constructor is `explicit` if and only if `std::is_convertible_v<Err, E>` is `false`.

```
template <class Err>
constexpr EXPLICIT unexpected(unexpected<Err> && e);
```

Effects: Initializes `val` as if direct-non-list-initializing an object of type `E` with the expression `std::move(e.val)`.

Throws: Any exception thrown by the selected constructor of `E`.

Remarks: This constructor participates in overload resolution unless

- `std::is_constructible_v<E, Err&&>` is `true`,
- `is_constructible_v<E, unexpected<Err>&>` is `false`,
- `is_constructible_v<E, unexpected<Err>&&>` is `false`,
- `is_constructible_v<E, const unexpected<Err>&>` is `false`,
- `is_constructible_v<E, const unexpected<Err>&&>` is `false`,

- `is_convertible_v<unexpected<Err>&, E>` is `false`,
- `is_convertible_v<unexpected<Err>&&, E>` is `false`,
- `is_convertible_v<const unexpected<Err>&, E>` is `false`, and
- `is_convertible_v<const unexpected<Err>&&, E>` is `false`.

This constructor is `explicit` if and only if `std::is_convertible_v<Err&&, E>` is `false`

..5.2.2 Assignment [*expected.unexpected.assign*] {#expected.unexpected.assign}

..5.2.3 Observers [*expected.unexpected.observe*] {#expected.unexpected.observe}

```
constexpr const E& value() const &;
constexpr E& value() &;
```

Returns: `val`.

```
constexpr E&& value() &&;
constexpr E const&& value() const&&;
```

Returns: `std::move(val)`.

..5.2.4 Swap [*expected.unexpected.ctor*] {#expected.unexpected.swap}

```
        void swap(unexpected& other) noexcept(see bellow);
```

Requires: `is_swappable_v<E>` is `true`

Effects: Equivalent to `using std::swap; swap(val, other.val);`.

Throws:

Remarks: The expression inside `noexcept` is equivalent to: `is_nothrow_swappable_v<E>` is `true`.

1.18. **.**.5.2.5 Equality operators [*expected.unexpected.equality_op*]

```
template <class E1, class E2>
    friend constexpr bool operator==(const unexpected<E1>& x, const unexpected<E2>& y);
```

Returns: `x.value() == y.value()`.

```
template <class E1, class E2>
    friend constexpr bool operator!=(const unexpected<E1>& x, const unexpected<E2>& y);
```

Returns: `x.value() != y.value()`.

..5.2.5 Specialized algorithms [*expected.unexpected.specalg*] {#expected.unexpected.specalg}

```
template <class E1>
    friend void swap(unexpected<E1>& x, unexpected<E1>& y) noexcept(noexcept(x.swap(y)));
```

Effects: Equivalent to `x.swap(y)`.

Remarks: This function shall not participate in overload resolution unless `is_swappable_v<E1>` is `true`.

1.19. **.**.6 Template Class `bad_expected_access` [*expected.bad_expected_access*]

```
template <class E>
class bad_expected_access : public bad_expected_access<void> {
public:
    explicit bad_expected_access(E);
    virtual const char* what() const noexcept override;
    E& error() &;
    const E& error() const&;
    E&& error() &&;
    const E&&  error() const&&;
private:
    E val; // exposition only
};
```

ISSUE 1 Wondering if we just need an `const &` overload as we do for `system_error`.

The template class `bad_expected_access` defines the type of objects thrown as exceptions to report the situation where an attempt is made to access the value of `expected` object that contains an `unexpected<E>`.

```
bad_expected_access::bad_expected_access(E e);
```

Effects: Initialize `val` with `e`.

Postconditions: `what()` returns an implementation-defined NTBS.

```
const E& error() const&;
E& error() &;
```

Returns: `val;`

```
E&& error() &&;
const E&& error() const &&;
```

Returns: `std::move(val);`

```
virtual const char* what() const noexcept override;
```

Returns: An implementation-defined NTBS.

§ 1.20. 𝟐𝟗.𝟐𝟗.7 Class `bad_expected_access<void>` [*expected.bad_expected_access_base*]

```
template <>
class bad_expected_access<void> : public exception {
public:
    explicit bad_expected_access();
};
```

§ 1.21. 𝟐𝟗.𝟐𝟗.8 `unexpect` tag [*expected.unexpect*]

```
struct unexpect_t {
    explicit unexpect_t() = default;
};
inline constexpr unexpect_t unexpect{};
```

§ 2. Open Questions

`std::expected` is a vocabulary type with an opinionated design and a proven record under varied forms in a multitude of codebases. Its current form has undergone multiple revisions and received substantial feedback, falling roughly in the following categories:

1. **Ergonomics**: is this *the right way* to expose such functionality?
2. **Disappointment**: should we expose this in the Standard, given C++'s existing error handling mechanisms?
3. **STL usage**: should the Standard Template Library adopt this class, at which pace, and where?

LEWG and EWG have nonetheless reached consensus that a class of this general approach is probably desirable, and the only way to truly answer these questions is to try it out in a TS and ask for explicit feedback from developers. The authors hope that developers will provide new information which they'll be able to communicate to the Committee.

Here are open questions, and questions which the Committee thinks are settled and which new information can justify revisiting.

§ 2.1. Ergonomics

1. Name:
   - Is `expected` the right name?
   - Does it express intent both as a consumer and a producer?
2. Is `E` a salient property of `expected`?
3. Is `expected<void, E>` clear on what it expresses as a return type?
4. Would it make sense for `expected` to support containing *both* `T` and `E` (in some designs, either one of them being optional), or is this use case better handled by a separate proposal?
5. Is the order of parameters `<T, E>` appropriate?
6. Is usage of `expected` "viral" in a codebase, or can it be adopted incrementally?
7. - Missing `expected<T,E>::emplace` for `unexpected<E>`.
   - The in-place construction from a `unexpected<E>` using the `unexpet` tag doesn't conveys the in-place nature of this constructor.
   - Do we need in-place using indexes, as variant?
8. Comparisons:
   - Are `==` and `!=` useful?
   - Should other comparisons be provided?
   - What usages of `expected` mandate putting instances in a `map`, or other such container?
   - Should `hash` be provided?
   - What usages of `expected` mandate putting instances in an `unordered_map`, or other such container?
   - Should `expected<T, E>` always be comparable if `T` is comparable, even if `E` is not comparable?
9. Error type `E`:
   - Have `expected<T, void>` and `unexpected<void>` a sense?
   - `E` template parameter has no default. Should it?
   - Should `expected` be specialized for particular `E` types such as `exception_ptr` and how?
   - Should `expected` handle `E` types with a built-in "success" value any differently and how?
   - `expected` is not implicitly constructible from an `E`, even when unambiguous from `T`, because as a vocabulary type it wants unexpected error construction to be verbose, and require hopping through an `unexpected`. Is the verbosity extraneous?

10. Does usage of this class cause a meaningful performance impact compared to using error codes?
11. The accessor design offers a terse unchecked dereference operator (expected to be used alongside the implicit `bool` conversion), as well as `value()` and `error()` accessors which are checked. Is that a gotcha, or is it similar enough to classes such as `optional` to be unsurprising?
12. Is `bad_expected_access` the right thing to throw?
13. Should some members be `nodiscard`?

§ 2.2. Disappointment

C++ already supports exceptions and error codes, `expected` would be a third kind of error handling.

1. where does `expected` work better than either exceptions or error handling?
2. `expected` was designed to be particularly well suited to APIs which require their immediate caller to consider an error scenario. Do it succeed in that purpose?
3. Do codebases successfully compose these three types of error handling?
4. Is debuggability any harder?
5. Is it easy to teach C++ as a whole with a third type of error handling?

§ 2.3. STL Usage

1. Should `expected` be used in the STL at the same time as it gets standardized?
2. Where, considering `std2` may be a good place to change APIs?

§ References

§ Informative References

[P0323r4]
Vicente Botet, JF Bastien. std::expected. URL: https://wg21.link/p0323r4

[P0323r5]
Vicente Botet, JF Bastien. std::expected. URL: https://wg21.link/p0323r5

§ Issues Index

ISSUE 1 Wondering if we just need an `const &` overload as we do for `system_error`.↵